

Homework05: Distributed SpMV Performance Analysis

David Haddad

Fall 2025

1 Overview:

In this report, I analyze the strong-scaling behavior of the COO-format sparse matrix-vector multiply (SpMV) using MPI present. Three experiments were run, with two being examined here for strong scaling analysis:

1. a single-node configuration using multiple MPI ranks, and
2. a multi-node configuration using one rank per node. All timings were obtained using `MPI_Wtime`. Speedup and efficiency were computed using the single-rank, single-node runtime $T_1 = 0.031262\text{s}$.

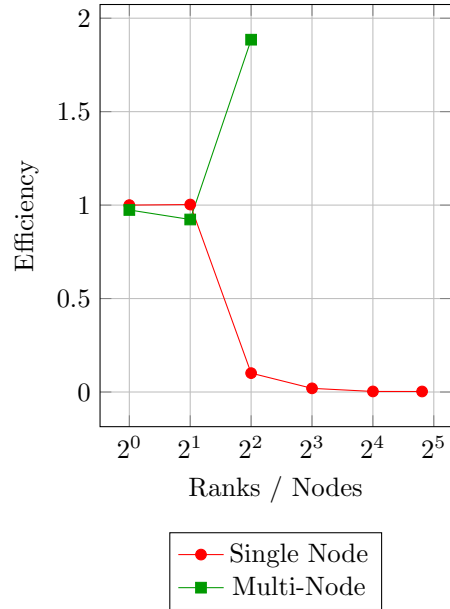


Figure 1: Parallel efficiency using $T_1 = 0.031262\text{s}$.

2 Results

2.1 Parallel Efficiency

Parallel efficiency is defined as

$$E_p = \frac{T_1}{pT_p}.$$

Figure 1 shows a complete collapse in efficiency for single-node MPI runs beyond two ranks. Lecture 16 (Slides 6–12) discuss how the irregular accesses can degrade the effective memory bandwidth through cache-line underutilization and the α - β (latency–bandwidth) model. This leads to severe performance penalties, as many cores compete for the same memory.

The distributed configuration, in contrast, shows near-linear and, at 4 nodes (the maximum number tested), even superlinear (kind of not really superlinear) numerical speedup behavior. This arises from cache-fit effects rather than algorithmic superlinearity. Since each rank receives a submatrix, the workset fits into the cache more easily (Lecture 16, Slides 6–15), reducing DRAM traffic and eliminating the atomic contention observed on a single node.

2.2 Execution Time Breakdown

How can we explain such a large performance gap? Figure 2 compares the 4-rank case on one node vs. four nodes. The time for the single-node run is dominated by atomic-update serialization. Lecture 16 explains how irregular write patterns lead to poor cache-line utilization and can create more memory traffic than the hardware can handle efficiently, causing the memory system to fall behind and waste time.

The four-node run avoids this shared-memory contention entirely. It has each rank work on its own submatrix, allowing their memory accesses to no longer interfere with each other. The communication phases (scatter and reduce) add overhead, but the drop in SpMV computation time amortizes this. Collective operations introduce a latency cost based on the depth of the communication tree, not the total volume of the data being sent (Lecture 14, Slides 8–26: Collective Communication).

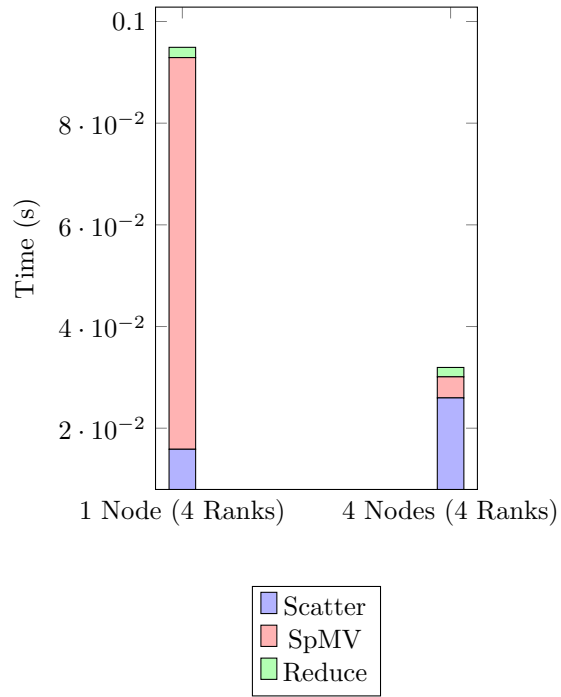


Figure 2: Execution breakdown for 4 ranks on one node vs. four nodes.

3 Conclusion

Single-node MPI performance degrades due to bandwidth limitations, cache-line underutilization, and serialization of atomic updates. Distributing the matrix computation across nodes enables easy parallelization in COO, reducing the working-set size per rank and effectively eliminating shared contention. The result is strong scaling and an apparent superlinear speedup (due to cache behavior rather than algorithmic superlinearity). This conclusion matches the locality and memory-hierarchy effects discussed in class.